

Functional Specifications

Abstract

This note gives a brief introduction to specification of functions. The specification is based on predicate logic to describe the contract between a function and its environment. The contract is in this note described as pre- and post-conditions. The reader has a well-founded knowledge of programming, and knowledge of predicate logic.

eaaa, Januar 2026
Ole Eriksen

Table of content

ABSTRACT	1
TABLE OF CONTENT	1
1. MOTIVATION	2
2. OUTLINE.....	2
3. SPECIFICATION.....	2
3.1 INFORMAL SPECIFICATION	3
3.2 FORMAL SPECIFICATION	4
4. IMPLEMENTATION	5
5. FINAL REMARKS.....	7
6. EXERCISES	7

1. Motivation

The motivation for this note is to give the professional programmer a set of concepts, techniques and best practice focusing on specification of functions.

The founder of the programming language c++ Bjarne Stroustrup has a quote: *"The most important single aspect of software development is to be clear about what you are trying to build."*¹. His point is that when you are facing a programming task, you can help yourself by being clear about what you are going to build before you start coding. In this note we will focus on specification of functions as a technique to achieve a clear understanding of what a function should do. We could call this for a specification-first approach.

The impact of starting to code before you have a clear and precise understanding of what you should build is buggy, complicated and ineffective code. Another approach is to spend a little time on making a specification for the code to be produced. This activity will focus directly on the most important aspect of software development, and free your mind for "how" questions before you have a clear and precise idea of "what". Being careful with the specification will imply less buggy and much simpler code.

Another way to achieve a clear understanding of what to develop is to construct the test for the function before making the implementation (often referred to as a test-first approach). This is also a very effective approach and can be combined with the specification-first approach.

Today's programmers use GAI to produce parts of their code. Therefore, competences in making specifications of high quality are crucial, because they will be the essential part of the prompts. In a near future GAI might turn the specification-first approach into a specification-only approach.

2. Outline

In section 3 the term specification is defined and further the scope is limited to so-called stateless functions. Section 3.1 describes informal specification. Section 3.2 and 4 are optional and dedicated readers with interest in more formal methods for making specifications and understand the link between the formal specification and implementation.

3. Specification

A functional specification is a description of a function focusing on *what*

¹ http://www.azquotes.com/author/14260-Bjarne_Stroustrup

the function offer. The specification can be regarded as a contract. A contract is an agreement between two parties; the function and its environment. Because of this asymmetry, we will denote the function as the server because it will offer functionality or service for its environment. To keep the terminology consistent, we will denote the environment as the client. Any agreement between the server and the client will include

- the service that is offered – in this note we will express this as a predicate and call it a post-condition, and
- what the client must fulfil before using the service – this is expressed as a predicate called the pre-condition.

The contract is therefore a description of the pre- and the post-condition. The pre-condition will typically be a constraint on the parameters of the function. The post condition will state the effect of the function. So, what is the effect of a function? Here we can distinguish between functional effect and side effect. The functional effect is the value of the results of the function including exceptions. The side effect is the modifications the function does on global state. A general best practice is to avoid side effects; it makes code complicated and very difficult to test. Therefore, we will in the following only deal with functions without side effects. In a following note we will dig into the specification of classes where side effects are valid and restricted to the instance variables – or state – of a class.

The functional specification can be more or less formal; the informal might communicate well but may lack of precision. The formal is very precise but may be hard to write and read.

3.1 Informal Specification

An informal specification of a function is a short plain text description of the effect of function. Most of the documentation of software libraries is informal specification. The example used throughout this note is four functions. An example of an informal specification for these could be described as:

```
interface IUsefulFunctions {  
  
    /* return true if p is a prime */  
    bool IsPrime(int p);  
  
    /* return the minimal value of a */  
    int Minimal(int[] a);  
  
    /* return the integer square root of x */  
    int IntSqrt(int x);  
  
    /* return true if a does not contain duplicates */  
    bool ContainDuplicates(int[] a);  
}
```

The interface is described informally, and therefore there might be issues that are not clear.

The reader is invited to consider any such issues with the four examples before reading further.

There are a few rules for making good informal specifications:

- Each parameter of the function should be mentioned at least once.
- It should be described what the function returns in all cases.
- There should be consistency between the return type of the function and the text.

In fact, none of the four examples fulfil these simple basic rules. The reader is invited to the problems and fix them by making specifications that fulfils the rules.

3.2 Formal Specification

As mentioned above, a formal functional specification is essential two logical predicates; the pre-condition describing constraints on input parameters, and the post-condition capturing the logical relation between the parameters and the result of the function. The examples from the former section formally specified could look like:

```
interface IUsefulFunctions {  
  
    /* Pre: true  
    /* Post:  
        result = p > 1 and  
         $\forall q: 1 < q < p \Rightarrow p \bmod q \neq 0$  */  
    bool IsPrime(int p);  
  
    /* Pre: a.Length > 0  
    Post:  $\exists i: 0 \leq i < a.Length$  and result = a[i] and  
         $\forall i: 0 \leq i < a.Length \Rightarrow a[i] \geq result$  */  
    int Minimal(int[] a);  
  
    /* Pre: x > 0  
    Post:  $result^2 \leq x$  and  $(result+1)^2 > x$  */  
    int IntSqrt(int x);  
  
    /* Pre: true  
    Post: result =  
         $\forall i: 0 \leq i < a.Length$  and  
         $\forall j: i < j < a.Length \Rightarrow a[i] \neq a[j]$  */  
    bool ContainDuplicates(int[] a);  
  
}
```

In this formal specification result is used as the result of the function.

The reader must now see if this formal specification removed some of the unclear issues embedded in the informal specification.

The reader is invited to work on exercise 1-5.

There are several benefits from making a formal specification:

It is a precise contract. In case of out sourcing the implementation to a colleague or another company (or even yourself) it gives the person that should do the implementation a very solid basic of what to construct.

It will make it perfectly clear what to implement. Even if you are going to do the implementation yourself, the process of making the formal specification will be an effective way to answer all the “what if” questions before you are going into details about the implementation.

Separation of concerns. When making a formal specification you are 100% concerned about what to build. Later your concern is how to build. In that way the formal specification will separate the “what” and the “how” in the programming process.

Test cases can be outlined before implementation. There are many recommendations about “test first” in literature, and the formal specification may be a very solid basic for writing test cases.

A final remark: if you have troubles making a formal specification, it might be because you really do not know what to build. And in that case, it will require a miracle that you end up with a correct implementation.

4. Implementation

After having made a clear and precise specification it is time for coding. There are several examples that a formal specification might help you with the implementation. There are more generic techniques supporting how to derive a correct implementation from a formal specification. But this will involve a more abstract concepts and techniques including a logic or semantic for the language. This is out of scope for this note, so in the following a few examples of how to “derive” a correct implementation from the specification are shown.

The first example is the computing a minimal value in an array. The post-condition is two parts: the first one

$$\exists i: 0 \leq i < a.Length \text{ and } result = a[i]$$

saying that the result must be present in the array, and the second part

$$\forall i: 0 \leq i < a.Length \Rightarrow a[i] \geq result$$

saying that the result of the function must be less than or equal to all values in the array.

The implementation can be a simple loop going through the entire array where a minimal value is updated properly. Because the pre-condition is that the array cannot be empty it is safe to initialize this minimal value to the first one (or last one).

The implementation in code could be:

```
int Minimal(int[] a)
{ if (a.Length == 0)
    throw new ArgumentException();
  int res = a[0];
  for (int i = 1; i < a.Length; i++)
    if (a[i] < res) res = a[i];
  return res;
}
```

The next example is the function computing the integer square root: The integer square root of an integer x , is the largest integer q , such that the square of q is less than or equal to x . The post-condition is

$result^2 \leq x$ and $(result+1)^2 > x$

where the first part says that the square of the result must be less than or equal to x , and the second part is saying it is the largest integer having this property. The implementation could be a simple iteration searching for this. We can start the iteration with 1, since the pre-condition will guarantee that the input is at least 1. This will lead to the following code:

```
int IntSqrt(int x)
{ if (x < 1)
    throw new ArgumentOutOfRangeException();
  int res = 1;
  while (res * res <= x)
    res++;
  return res - 1;
}
```

The reader is invited to implement some of the functions from exercise 1-5. Try to derive the implementation from the formal specification by using the following rules:

- a) Throw exceptions when pre-condition is not meet
- b) A universal quantifier in the post-condition might imply an iteration in the implementation.

5. Final Remarks

The scope of such a detailed specification is limited. There are many examples, where formal specification will not add any essential benefits to the programming process. But in case you can imagine several different ways to implement a class the formal specification may be beneficial.

6. Exercises

In the following exercises, you are supposed to construct informal and formal specification for functions. In all exercises, you are invited to see if the specification can be used as a constructive hint for an implementation.

Exercise 1: Make a specification (both an informal and formal) for a boolean function returning true if an array is increasing.

Exercise 2: Make a specification (both an informal and formal) for a boolean function returning true if the input parameter is a cube-number.

Exercise 3: Make a specification (both an informal and formal) for an integer function returning the minimal value of an array.

Exercise 4: Make a specification (both an informal and formal) for a function that can be used to validate a date. The function will have three input parameters: day, month, and year. The function must return true if input represents a valid date.

Exercise 5: Make a specification (both an informal and formal) for a function that can reverse an array of integers.